This is a league based challenge.

For this challenge, multiple leagues for the same game are available. Once you have proven your skills against the first Boss, you will access a higher league and extra rules will be available.

In the first few leagues, your submission will only fight the boss in the arena. Win a best-of-five to advance.

Goal

The first two Leagues have a special **objective** to achieve. The full game will play, but you can only win by completing the objective. Once you do, you can start work on your complete bot.

League Objective 1:

Connect two **towns** to form an **active train connection** to instantly win the game.

The **Boss** AI will skip their turns. If you fail to form a connection within `100` turns, you will lose. Win at least `3` or more times out of `5` to progress to the next league.

Rules

In this game both players use **paint** to draw train tracks on a **magic** map, connecting towns on the map will bring prosperity to your own world.

The map is represented in the game by a **grid**.

Map

The grid is made up of cells that can have one of three types:

- Type `0` for **plains**.
- Type `1` for **river**.
- Type `2` for **mountains**.

The grid is partitioned into **regions**. Each region is made up of multiple contiguous cells. Each region has a unique `regionId`. Regions are susceptible to **disruption** by players. *More information on disruption in next league.*

Some regions will contain a **town**. Towns can only be found on **plain** cells.

Towns

Each game starts with multiple **towns** placed randomly across the map. There will only be one per **region** and no two regions sharing a border will both contain a town.

Each town has a unique `townId`.

Each town will have a list of `desiredConnections`: a list of town ids representing all the **other towns** this town would like to be **connected to** via **train tracks** placed by players.

Providing a town with **train tracks** connecting it to a desired town is how players score **points**.

Desired connections are **unilateral**. Meaning if town `0` spawns with a desired connection to town `1`, town `1` **will not** want to connect to town `0`.

A town can have **zero** `desiredConnections`, but will always be the subject of at least one other town's `desiredConnections`.

Placing Train Tracks

Players are given on each turn `3`
paint points they can use to place **train tracks** on the map. ⚠️ These points do not carry over to the next turn and will be lost if left unused.

It costs:

- `1` paint point to place a track on **plains**.
- `2` paint points to place a track on **river**.
- `3` paint points to place a track on **mountains**.

A track's `owner` is the `playerId` (`0` - `1`) of the player that placed it. They will be the same color.

If both players place a track on the **same turn** at the **same location**, the track's `owner` will be `2` , indicating a neutral track piece.

A train track **cannot** be placed on a **town** or on an existing track.

Once placed, a train track will **automatically connect** to other **tracks** and **towns** orthogonally adjacent to it.

Connections

For each pair of towns in which one has the other in its `desiredConnections`, if at least one **path** between the two exists, the shortest such **path** becomes the **active connection** between those towns.

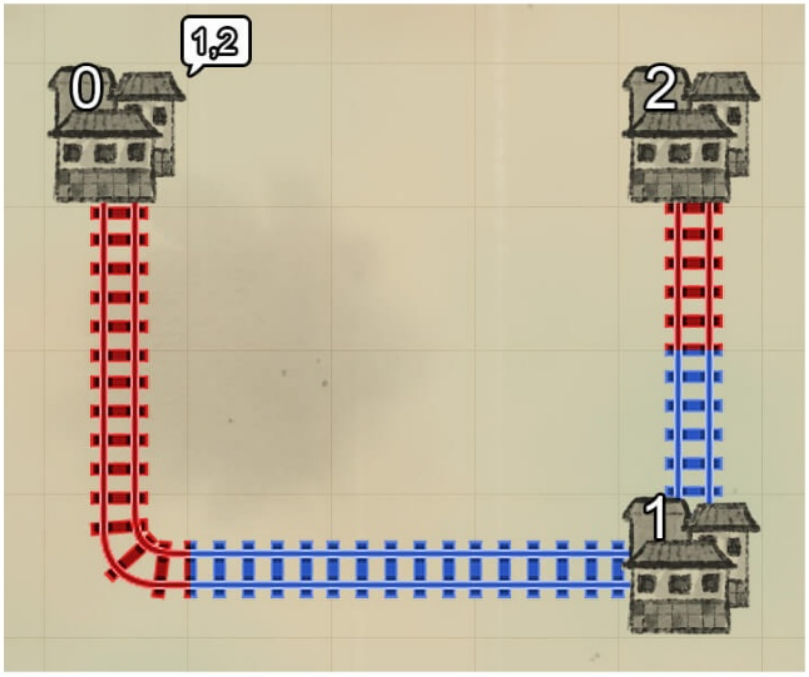
A **path** is an uninterrupted sequence of orthogonally adjacent cells with a **train track** or a **town**.

If there are **multiple** shortest paths, the chosen path will always **prioritize** the direction in the following order when moving from the requesting town to the desired connected town:

1. `NORTH`
2. `EAST`
3. `SOUTH`
4. `WEST`

At the end of every turn, each **active connection** will provide `1` point to each player for every track they **own** in the **path**.

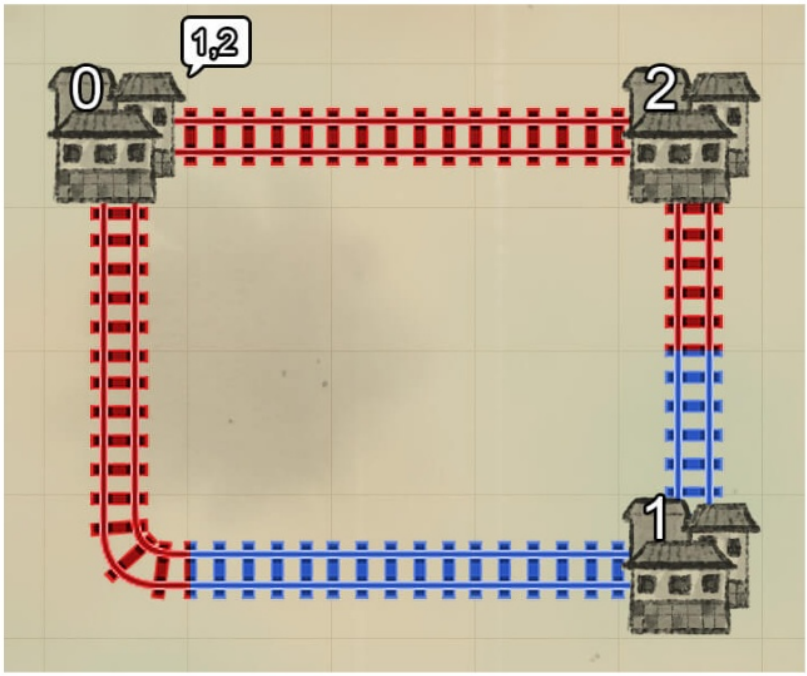
Example 1:



Here, there is an **active connection** from town **0** to town **1** and one from town **0** to town **2**.

Both the red and the blue players will gain `3` points for the **0-1** connection and `4` points for the **0-2** connection at the end of the turn.

Example 2:



Here, only the shortest path from **0** to **2** is used for connection **0-2**. Meaning the red player will now gain `3` points for that connection, and the blue player will get none.

There are two equally short paths from **0** to **1**, but since `EAST` has a higher priority than `SOUTH` , the chosen path will be the one that goes through town **2**. The red player will gain `4` points for that connection, and the blue player will only gain `1` .

🎬 Actions

Each turn, players must provide at least one action on the standard output.

Actions must be separated by a semicolon `;` and be one of the following:

- `PLACE_TRACKS x y` : place a track on a free cell.
- `AUTOPLACE fromX fromY toX toY` : automatically generates a list of actions for the **cheapest** path from `from` to `to` in terms of paint points. This will do nothing if a path already exists.
  The generated actions replace this command.
- `WAIT` : do nothing.

Victory Conditions

Form an **active connection** and score any number of points.

Defeat Conditions

- Your program does not provide a command in the allotted time or one of the commands is invalid.
- You do not complete the objective within `100` turns.

Debugging tips

- Hover over the grid to see extra information on the cell under your mouse.
- Press the gear icon on the viewer to access extra display options.
- Use the keyboard to control the action: space to play/pause, arrows to step 1 frame at a time.

Click to expand

Game Protocol

Initialization Input

To Start

Why not start the battle with one of these **AI Starters**, provided by the team:

- C++ https://gist.github.com/CGjupoulton/bbd4b720ae7e0f1e5f2e970bb42ed066
- JavaScript https://gist.github.com/CGjupoulton/279a0f6ce4995bf4c55391fd40ae8bff
- Java https://gist.github.com/CGjupoulton/17397683833e324a7b8c7c7c642239d3
- Python https://gist.github.com/CGjupoulton/5a73bbd1142af98c6ca6887648b07cc2

You can modify them to match your style or take them as an example to code everything from scratch.